Application of the Rijndael Field to Hashing

Kieran Harvie

Copyright ©December 12, 2023. All Rights Reserved.

Contents

1 Introduction **3**
1.1 Structure of this Document . . . 3
1.2 Structure of the Repository . . . 3

2 Theory **4**
2.1 Hashing 101 . . . 4
2.1.1 Hashing Terminology . . . 5
2.2 Abstract Algebra 101 . . . 5
2.3 Field Theory 101 . . . 6
2.3.1 Constructing Fields . . . 8
2.3.2 Finite Fields . . . 9
2.4 The Rijndael Field . . . 11
2.4.1 Bytes and Notation . . . 12
2.4.2 Generators . . . 14

3 Matrix Methods **15**
3.1 Gaussian Elimination . . . 15
3.1.1 (pointlessly) Abstract Algebra Preamble . . . 15
3.1.2 (useful) Matrix Preamble . . . 15
3.1.3 The Algorithm . . . 17
3.2 Application to Hashing . . . 17
3.2.1 Example . . . 18
3.2.2 Collision . . . 19
3.2.3 Choice of a_i . . . 19
3.2.4 Choice of f_j . . . 19

4 Polynomial Methods **21**
4.1 Interpolation . . . 21
4.1.1 Lagrange Interpolation . . . 21
4.1.2 Vandermonde Matrix . . . 23

4.2 Evaluation 24
4.2.1 Naïve Scheme 24
4.2.2 Horner's Scheme 25
4.2.3 Estrin's Scheme 26
4.3 Application to Hashing 26

5 Advanced Encryption Standard 27

A Appendix 28
A.1 Ring Theory 101 28
A.1.1 Commutative Rings 29
A.1.2 Polynomial Rings 29
A.1.3 Bézout's Identity and Domains 31
A.1.4 Conclusion 34
A.2 Adjoining a Number to a Field 34
A.3 Generator Tables 35
A.4 Side-Channel 37
A.5 Matrix Arithmetic 38
A.5.1 Diagonalization 38
A.5.2 Vandermonde Polynomial 39
A.6 Hardware and Instruction Sets 41
A.6.1 CLMUL Instruction Set 41
A.6.2 AES Instruction Set 41

Introduction

General introduction This is an accompanying document to the repository.

1.1 Structure of this Document

It is meant to approachable to someone that has freshman understand of math or programing, while possibly requiring some external research to use to the fullest. And hopefully being giving enough of a taste to inspire that research.

And maybe even an ambassador for abstract algebra and cryptography for people that are interested in those things but need practical applications to make them stick.

There is a bit of a hike to get towards the Rijndael Field and people familiar with abstract algebra can skip the rest of this introduction.

1.2 Structure of the Repository

Theory

2.1 Hashing 101

For our purposes a hash function is an any function that inputs a string and outputs a number. The most direct example is converting the first character to it alphabetical number (A=1, B=2, etc.):

$$\text{hash(Apple)} = 1, \quad \text{hash(Orange)} = 15$$

There are two main applications for hashing: The first is performance, as operations on strings may be more expensive then on numbers so an efficient and well designed hash may allow us to work better on the numbers instead. The second is cryptography, that is the practice of communicating secrets, where we want to hash the string and communicate the number in hopes of keeping the string secrete.

There is a clear tension between these two applications. For performance you'd want as much information about the initial string easily assessable in the hash. But for cryptography you'd want as much information hidden as possible while still allowing communication.

This document aims to serve as an introduction, so wont optimize for either application, but I bring them up this early to wet the readers appetite about why they might care about hashing. **But also to state that the reader should not apply any algorithm here for cryptographic purposes without consulting a professional first.** At least one technique used makes the algorithm susceptible to a side-channel attack, and even if I avoided that technique there would be another one I'm ignorant of.

2.1.1 Hashing Terminology

With applications and warning asides there are some more things the reader should know. First is that in common use the term hash can mean both the hash function and the resulting number from that function.

Secondly there are four general terms worth knowing:

Collision: A collision is when two strings have the same hash. This is broadly undesirable since any operation we want to preform on the hash has to work for both strings.

Hash Table: A common application of hash functions in environments where accessing a look-up table with string keys are inefficient compared to number keys. We basically make a look-up table with number keys and when asked to look-up a string key we hash it first and use the result as the integer key. This will require us to deal with collisions as they come up.

Perfect Hash Function: A perfect hash for a given set of strings is hash function where none of strings collide.

Minimal Perfect Hash Function: A minimal perfect hash for a given set is a perfect hash function where all outputs of the string are less then or equal to the size of the set. In terms of features, but not necessarily performance, this is the ideal hash function as there are no collisions to deal with and no wasted numbers taking up space.

2.2 Abstract Algebra 101

A two sentence summary of Abstract Algebra would be something like:

> Abstract Algebra is the study of Algebraic Structures.
>
> Algebraic Structures are a set of objects (e.g. Numbers) and operations (e.g. $+, \times, -, \sqrt{\ }$) that follow certain axioms (e.g. $a + b = b + a$).

One thing I want to stress is that from the abstract algebra viewpoint it is taken as an axiom that $a + b = b + a$.

While one could make arguments about why $1 + 2 = 2 + 1$ in abstract algebra it's just kind of assumed as an axiom in algebraic structures of numbers. And we instead focus figuring out what are the logical conclusions of all the assumed axioms.

The reason this is a useful thing to do is that it enables us reuse arguments for other structures given they have the same axioms used in the argument. Consider a proof of the difference of two squares:

$$\begin{aligned}(a-b)(a+b) =& a(a+b) - b(a+b) \\ =& (a^2 + ba) + ((-b)a + (-b)b) \\ =& a^2 + (ba + ((-b)a) + -b^2 \\ =& a^2 + a(b + (-b)) - b^2 \\ =& a^2 + a \times 0 - b^2 \\ =& a^2 - b^2\end{aligned}$$

We can list all the axioms used[1] and apply know that the difference of two squares is the natural conclusion of these axioms. And hence apply this formula to any algebraic structure that follows the axioms used.

2.3 Field Theory 101

> Fields are Algebraic structures that act like the rational numbers $\mathbb{Q}$ (i.e. Fractions with $+, -, \times, \div$)

There are a lot of axioms for a field, and I don't think listing them all here would be useful. Especially when the reader can get the gist with an example.

Consider multiplication and addition of integers less then 5 where if the answer is greater than 5 we divide it by 5 and take the remainder:

$+$	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

$\times$	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

Here are some interesting properties of these tables:

- They are mirrored around the diagonal. Meaning $a + b = b + a$ and $ab = ba$.

[1] e.g. $a(b+c) = ab+ac$, $ab = ba$, $a+b = b+a$, $0 \times a = 0$, $b+(-b) = 0$, $(-b)b = -b^2$, ...

- The number 0 has the properties $0 + a = a$ and $0 \times a = 0$.
- The number 1 has the property $1a = a$.
- For every number a there is a unique number b such that $a + b = 0$. (Meaning we can define $-a = b$).
- For every nonzero number a there is a unique number b such that $ab = 1$. (Meaning we can define $\frac{1}{a} = b$).

These fact combined means we can define and manipulate fractions in this structure the same way we would in $\mathbb{Q}$. (Using $\frac{a}{b} = a \times \frac{1}{b}$ as needed). These operations (Addition, Multiplication, Negation, and Reciprocation) are called the field operations and the fact we can do all of them makes "Integers less then 5 where we can take the remainder of any operation"[2] a field.

I said one of the benefits of abstract algebra is the reuse of arguments where the axioms still apply. So lets test this by applying the difference of squares proof from the previous section to 2 and 4:

$$4^2 - 2^2 = 1 - 4 = -3 = 2$$

and

$$(4 - 2)(4 + 2) = 2 \times 1 = 2$$

Which are equal, as expected.

It's also worth noting what we can't do in this structure:

- Square root: Notice that in the $\times$ table diagonal there is no 2 or 3 this means we can't meaningfully talk about $\sqrt{2}$ or $\sqrt{3}$.
- Order: Notice that $4 + 1 = 0$ but that normally we would say $0 < 4$ however adding 1 to both sides gives $1 < 0$.

That's fine because neither of these concepts are part of the field axioms and this neutrality is what makes abstract algebra so powerful.

Despite not being part of the field axioms, order makes sense in $\mathbb{Q}$, and square root makes sense for positive numbers in $\mathbb{R}$. This is because axioms in abstract algebra only guarantee the structure they are meant to describe

[2] For reasons that will become clear later this structure is called $GF(5)$

and don't prohibit additional structure. You can pick and choose axiom and how they interact and take them to their logical conclusions while not having to repeat work what only requires certain subset of the axioms.

We already implicitly took advantage of this power at the start when we defined field as like $\mathbb{Q}$ but not $\mathbb{R}$. Operations like square roots working for all positive integers aren't going to be important for our hashing application so we are using a field to work because that's all we need.

2.3.1 Constructing Fields

To really use the full potency of Field Theory, and eventually use the Rijndael field, we need to move past describing fields that we already know and move to constructing new fields.

The most obvious way to construct a field is to take an existing field F and add another number a to it while doing whatever is necessary to make sure the field axioms still apply. This process is called the adjoining of a to F.

As observed earlier there is no $\sqrt{3}$ in the field F of integers remainder 5.

What if we define a new set G such that:

$$G = \left\{ a + b\sqrt{3} \,|\, a, b \in [0, 4] \right\}$$

With addition, multiplication, and negation working as expected (taking the remainder as needed):

$$\begin{aligned}
(a_0 + b_0\sqrt{3}) + (a_1 + b_1\sqrt{3}) =& (a_0 + a_1) + (b_0 + b_1)\sqrt{3} \\
(a_0 + b_0\sqrt{3}) \times (a_1 + b_1\sqrt{3}) =& (a_0b_0 + 3a_1b_1) + (a_0b_1 + a_1b_0)\sqrt{3} \\
-(a_0 + b_0\sqrt{3}) =& (-a_0) + (-b_0)\sqrt{3})
\end{aligned}$$

And reciprocation defined as:

$$\frac{1}{a + b\sqrt{3}} = \frac{1}{a^2 - 3b^2}(a - b\sqrt{3})$$

Hence we have created a new field G that contains F as a subfield, when $b = 0$ and includes a square root of 3 while satisfying of the field axioms.

The theory for constructing new fields was laid by groundbreaking work 19th mathematician Évariste Galois. Like much groundbreaking work before and after it wasn't understood in it's time. With influential mathematicians like Cauchy rejecting Galois' papers and Poisson calling it "incomprehensible". This lack of understanding was not helped when Galois' political activism landed him in jail after the French Revolution of 1850 and upon being release immediately entering a duel and dying.

Surly such a groundbreaking idea from such a interesting life must be too complex for regular people to understand? Well what this groundbreaking work focused on was finding the roots of polynomials, something you likely did in high school.

Because multiplication and addition are field operations given a field F we can define the set of polynomials with coefficients in F as $F[X]$. And if a can be written as the solution to a polynomial in $F[X]$ we can create a new field $F(a)$ where the elements of the new field are polynomials in the old field evaluated at a, this set is denoted $F[a]$.

This process is the same as the one shown in the beginning of the section where the polynomial used there was $X^2 - 2$

As abstract as this all seems you have likely done this implicitly before if you've ever used complex numbers. You were likely told to treat $\mathbb{C}$ and $\mathbb{R}$ but with an extra number i such that $i^2 = -1$. You can now see that $\mathbb{C}$ and $\mathbb{R}$ are fields and that $\mathbb{C}$ is what you get when you follow this adjoining process on $\mathbb{R}$ with $X^2 + 1$.

2.3.2 Finite Fields

One final piece of theory before the Rijndael Field is about the special case where a field as has finite number of elements.

These finite fields are also called Galois Fields, in honor of Galois, and a finite field of size n is denoted GF(n) or $\mathbb{F}_n$. The field of "Numbers less than 5 where we take the remainder after operations" that we have used had the proper name is GF(5) or $\mathbb{F}_5$ and the and when adjoined with $\sqrt{3}$ it would become GF(5)($\sqrt{3}$) or $\mathbb{F}_5(\sqrt{3})$.

There is a lot of interesting facts we know about finite fields, but perhaps

a better word would be restrictions.

Eagle eye readers may have already figured out one of these restrictions from the previous notation. How can GF(5) distinguish between two different finite fields of size 5? Well they don't have to because all finite fields of the same size are "isomorphic" with each other. Which is an abstract algebra way of saying you can change what you label everything ($1 \to a$, $2 \to \triangle$...) but the structure is the same.

Those multiplication tables where the only possible multiplication tables for five elements that are also a field. And since all the structure came from those tables it doesn't matter what we call the elements.

I don't know of any easy proof/argument for this so I'm going to have to leave it as out of scope, but readers are encouraged to do their own research.

This means that $\mathrm{GF}(5)(\sqrt{3}) = \mathrm{GF}(25)$, which cleans up the ugly notation on the right hand side.

Another interesting restriction on structure is that not all n can have a finite field. Only n that are the power of some prime p can have a finite field.

The intuition as to why prime numbers might be important consider the constructing a field of size pq, where p and q are different primes, like we constructed the field of size 5.

For this set to be a field there needs to be a reciprocal of p, meaning an x such that px has remainder 1 when divided by pq. Now we can assume $x < q$ because if $x > q$ we take $x - q$ as the new x since $p(x - q) = px - pq$ which has the same remainder when divided by pq and we can repeat this step as much as we need until $x < q$. But if $x < q$ then $px < pq$ meaning we don't have to take the remainder when calculating px meaning $px = 1$, no extra step needed. But $p > 1$ and $x \geq 1$ meaning $px \geq 1$ which contradicts $px = 1$, hence there can be no reciprocal of p in a field of size pq constructed this way.

This argument actually only shows we can't construct a field of size pq the way we constructed 5, but the gist that there is a problem with reciprocals when we mix primes remains.

While not useful to for out treatment of the Rijndael Field there is one

more cool property of finite fields. That for the field $\mathrm{GF}(p^n)$ we have:

$$(a+b)^p = a^p + b^p$$

No binomial expansion needed.

2.4 The Rijndael Field

The Rijndael Field is another name for $\mathrm{GF}(2^8)$ and first entices the math savvy programmer by it's size. 2^8 is the same as the number of states in a byte, and there are a tone of useful algorithms for $\mathbb{R}$ and $\mathbb{Q}$ that only rely on field operations. So can these algorithms become useful if applied to bytes through the Rijndael Field?

Well the best answer is to check, and the best way to check to to construct $\mathrm{GF}(2^8)$.

Lets start with the multiplication and addition tables for $\mathrm{GF}(2)$:

$+$	0	1
0	0	1
1	1	0

$\times$	0	1
0	0	0
1	0	1

Clearly $+$ is the XOR operator and $\times$ is the AND, already promising for computational applications.

Now I present, without explanation, the following element of $\mathrm{GF}(2)[X]$

$$p(X) = X^8 + X^4 + X^3 + X + 1$$

And assert that it constructs $\mathrm{GF}(2^8)$ when used in the same way as in the construction section of Field Theory 101. This can be proved rigorously with Galois Theory but is outside the scope of this document. And for our purposes all that matters is that it does work and generates a representation of $\mathrm{GF}(2^8)$:

$$\mathrm{GF}(2^8) = \{a_0 + a_1X + a_2X^2 + a_3X^3 + a_4X^4 + a_5X^5 + a_6X^6 + a_7X^7 \mid a_n \in [0,1]\}$$

Addition and multiplication follow normal polynomial arithmetic, remembering that the coefficients are in GF(2^8):

$$\begin{aligned}
(1+X^2+X^5+X^7)+(X^3+X^5) =& 1+X^2+X^3+(1+1)X^5+X^7 \\
=& 1+X^2+X^3+X^7 \\
(X^2+X^3)(1+X^5) =& 1+X^2+X^3+X^5+X^8 \\
=& 1+X^2+X^3+X^5+(p(X)-X^4-X^3-X-1) \\
=& -X+X^2-X^4+X^5+p(X) \\
=& X+X^2+X^4+X^5+p(X) \\
=& X+X^2+X^4+X^5
\end{aligned}$$

(Note that the last line comes from how GF(2^8) was constructed through $p(X)$ means $p(X)=0$).

This is very promising as we can convert an element of GF(2^8) to a byte by reading off coefficients from a 7th degree polynomial. Addition in GF(2^8) is just bitwise XOR. And Multiplication is a convolution where we trim of element greater than X^8 by using $p(X)=0$.

Multiplication is already a computationally tractable but there is a better method using "Generators". So with simple addition, easy conversion between GF(2^8) and bytes, and the promise of a better multiplication I hope you are convinced that the Rijndael Field is computationally promising.

2.4.1 Bytes and Notation

Carrying Xs around is annoying, especially since the coefficients can only be 1 or 0 meaning they are either there or not. We have previously stated the main allure of the Rijndael Field is that it has a one to one mapping to bytes. So maybe writing them as bytes will be more convent.

Want a Byte?

A bit is something that can either be on (1) or off (0) and a byte is eight[3] bits.

[3]Technically A byte is the smallest addressable unit of addressable memory in a computer. But society has all but agreed that this is eight bits.

So we can represent a byte as a list of eight 1s or 0s. For example, a byte where the first four bits are on and the rest off would be written as 11110000.

To make this more compact we interpret this list as a binary number and then write that number in hexadecimal[4].

Decimal	Binary	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

So the previous half on byte would be $F0$.

For a more worked example consider figuring out how the number 166 would be stored in a byte:

$$166 = 2^7 + 2^5 + 2^2 + 2^1 \to \overbrace{1010}^{A}\overbrace{0110}^{6} = A6$$

Or we could equivalently use hexadecimals directly:

$$166 = 10 \times 16^1 + 6 \times 16^0 \to A6$$

The final piece is to observe that each of the bits of a byte can represent one of the coefficients of a polynomial in $\mathrm{GF}(2)[X]$:

$$X^7 + X^4 + X^2 + 1 \to \overbrace{1001}^{9}\overbrace{0101}^{5} = 95$$

[4]Base 16

2.4.2 Generators

A generator for a field is an element g such that for any non-zero element a there is some integer n such that:

$$a = g^n$$

Where exponentiation is just iterative multiplication, nothing special.

Generators are useful because they simplify the multiplication of field element. For example, the generator for the Rijndael field is $X^2 + 1$ and we wish to multiply $X^2 + 1$ and $X^4 + 1$. We have:

$$X^2 + 1 = (X + 1)^2, \quad X^5 + X^4 + X + 1 = (X + 1)^5$$

To do this the direct way we would need to preform polynomial convolution and then cancel any relevant terms. But knowing they are powers of $X + 1$ we can instead use:

$$\begin{aligned}(X^2 + 1)(X^5 + X^4 + X + 1) =& (X + 1)^2(X + 1)^5 \\ =& (X + 1)^7 \\ =& X^7 + X^6 + X^5 + X^4 + X^3 + X^2 + X + 1\end{aligned}$$

This looks promising, but expanding $(X+1)^7$ still looks tiresome. But we can have a table of powers of $X + 1$ and just look up the 7th item. Combine this with a table which coverts a polynomial to which power of $X + 1$ it is, called the index, and we can stream line multiplication to three lookups (two indexes, one power).

These tables would only require 512 bytes and have been included in the appendix.

For brevity the tables write the elements in byte form so if you wish to verify the math of this section would need:

$$\begin{aligned}X + 1 &\to 03 \\ X^2 + 1 &\to 05 \\ X^5 + X^4 + X + 1 &\to 33 \\ X^7 + X^6 + X^5 + X^4 + X^3 + X^2 + X + 1 &\to FF\end{aligned}$$

Matrix Methods

From an abstract algebra a matrix is actually a type of tensor as field[it the underlying object is just a ring it's called a module. Say how walking through a 3×3 examples is useful to show only field operations is used. set spell not working?

MAKE SURE TO USE THE APPENDIX!!

3.1 Gaussian Elimination

3.1.1 (pointlessly) Abstract Algebra Preamble

From an abstract algebra viewpoint, the choice of dual space basis is arbitrary provided they have the right span. We can get such a new dual basis $\mathbf{e}'_{\mathbf{i}}$ from an old basis $\mathbf{e}_{\mathbf{j}}$, provided T is non-singular, through:

$$\mathbf{e}'_{\mathbf{i}} = T(\{\mathbf{e}_{\mathbf{j}}\})$$

So we can transform relationship between the coefficients of tensors in one basis it to another, and if T is affine then the new coefficients are directly transformed through T.

Three particular choices of T are swapping two the basis elements, multiplying a basis element by a non-zero constant, and adding a multiple of basis element index to another.

3.1.2 (useful) Matrix Preamble

Abstract tensor analysis is cool, for the type of people that like that type of thing, but the useful result of the previous subsection is that for matrices a,

b, and M, plus an invertible matrix T we have:

$$M_i^j a^i = b^i \Leftrightarrow T_k^j M_i^k a^i = T_k^j b^k$$

(And this result can be verified directly by multiplying by T or T^{-1} and short-circuiting the previous preamble).

Bellow are examples of the three previous particualr choices of T, hence forth referred to as elementary row operations, for 3×3 matrices:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{31} & m_{32} & m_{33} \\ m_{21} & m_{22} & m_{23} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & a & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ am_{21} & am_{22} & am_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & a & 1 \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} + am_{21} & m_{32} + am_{22} & m_{33} + am_{23} \end{bmatrix}$$

Since these elementary row operations can be represented through matrices their composition can be represented through matrix multiplication. Hence we can build more complex row operations from multiplying a chain of elementary row operations together.

For example say we want a transform T that takes the a 3×3 matrix and makes it first column $[1, 0, 0]$. Assuming $m_{1n} \neq 0$, we have:

$$\begin{bmatrix} m_{11}^{-1} & 0 & 0 \\ 0 & m_{22}^{-1} & 0 \\ 0 & 0 & m_{33}^{-1} \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} 1 & m_{12}m_{11}^{-1} & m_{13}m_{11}^{-1} \\ 1 & m_{22}m_{12}^{-1} & m_{23}m_{12}^{-1} \\ 1 & m_{32}m_{32}^{-1} & m_{33}m_{31}^{-1} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & m_{12}m_{11}^{-1} & m_{13}m_{11}^{-1} \\ 1 & m_{22}m_{12}^{-1} & m_{23}m_{12}^{-1} \\ 1 & m_{32}m_{32}^{-1} & m_{33}m_{31}^{-1} \end{bmatrix} = \begin{bmatrix} 1 & m_{12}m_{11}^{-1} & m_{13}m_{11}^{-1} \\ 0 & m_{22}m_{12}^{-1} - m_{12}m_{11}^{-1} & m_{23}m_{12}^{-1} - m_{13}m_{11}^{-1} \\ 0 & m_{32}m_{32}^{-1} - m_{12}m_{11}^{-1} & m_{23}m_{31}^{-1} - m_{13}m_{11}^{-1} \end{bmatrix}$$

Hence:

$$T = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} m_{11}^{-1} & 0 & 0 \\ 0 & m_{22}^{-1} & 0 \\ 0 & 0 & m_{33}^{-1} \end{bmatrix}$$

Gaussian Elimination is simply using this process to find a matrix T such that $T(M) = I$ by interatvly zeroing the outer, non-diagonal, elments:

$$\begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & m'_{22} & m'_{23} \\ 0 & m'_{32} & m'_{33} \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & m''_{33} \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

This calculates M^{-1} as the product of matrices representing elemetry row operations. Note that this always works when M is invertable, adafsd but we may need to swap row such that errent zeros put directly into their destination to avoid dividing by zero.

Rrwrite to bring in the opening. Jordan normal form??

3.1.3 The Algorithm

Matrices can easily be represented as 2d-arrays, elementary row operations aren't hard to implement either, and the iterative nature of final equation directly suggests the structure of the algorithm. However there is one more observation instead of keeping in memory all the elementary row operation, just directly apply them to the B as we go. Augmented matrices are excellent at facilitating this.

3.2 Application to Hashing

Consider an array of n strings, s_i, and an array of n values, a_i. Let their be some function f that converts a string to n field elements. Let $m_{i,j}$ be the jth field element for string s_i. Construct the matrix M from $m_{i,j}$ and the column vector A from a_i in the unusual ways. If M is invertible, then we can use the above algorithm obtains a column vector B such that:

$$MB = A$$

Which in element form is:

$$\sum_{j=1}^{n} b_j f(s_i)_j = a_i$$

Which defines the hash function:

$$h(s) = \sum_{j=1}^{n} b_j f(s)_j$$

Such that:

$$h(s_i) = a_i$$

As required.

3.2.1 Example

Consider the strings:

$$s_1 = \text{"APPLE"}, \quad s_2 = \text{"ORANGE"}$$

The hash values:

$$a_1 = 01, \quad a_2 = 02$$

The expression $f(s_i)_j$ maps the jth character in the ith string to it's place in the alphabet. Relevant conversions done bellow, full conversion can be found in the theory section:

Character	Decimal	Field Element
A	1	01
O	15	0A
P	16	10
R	18	12

Hence the requirement for B is such:

$$\begin{bmatrix} 01 & 10 \\ 0A & 12 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} 01 \\ 02 \end{bmatrix}$$

Which has solution:

$$\begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} 18 \\ F8 \end{bmatrix}$$

And gives the hash function:

$$h(s) = 18 \times f(s)_1 + F8 \times f(s)_2$$

3.2.2 Collision

We want to know when two strings s_0 and s_1 collide. Assume at least one of b_j is nonzero, else $h(s) = 0$ for all s. Then, without loss of generality, Assume that $b_n \neq 0$. Then expansion of definitions shows that $h(s_1) = h(s_0)$ if and only if:

$$f(s_1)_n = f(s_0)_n + b_n^{-1} \sum_{j=1}^{n-1} b_j (f(s_0)_j + f(s_1)_j)$$

If we assume that $f(s)_n$ has a uniform distribution, then the chance of colliding is $|GF(2^8)|^{-1} = 2^{-8}$.

3.2.3 Choice of a_i

Note that the example choice of a_i makes h as perfect minimal hash. But we can choose the values of a_i for other functions too, for example classification.

s_i	a_i
Apple	0
Orange	1
Pear	0
Lemon	1

$h(s)$ is now a boolean classification function that indicates whether or not the fruit is citrus of not.

3.2.4 Choice of f_j

The example choice of $f(s)_j$, where $f(s)_j$ is the jth character of s, has obvious shortcomings, consider the set:

$$S = \{\text{APPLE}, \text{APRICOT}\}]$$

M fails to be invertible because all string share the same first two characters and hence M has a duplicate row.

$$M = \begin{bmatrix} 01 & 10 \\ 01 & 10 \end{bmatrix}$$

This is an unfortunately common problem to encounter when S is composed of natural language strings. Not only do such strings contain patterns which cause an nonuniform distribution of characters, alternating constant and vowel clusters for example, but in many application the strings will share entire morphines at the beginning.

For example, imagine making a hash function in a parser that is meant to convert strings representing command to a jump table index. It is possible that multiple commands will start with "set_" or "get_".

But duplication alone underestimates the size of the shortfall. For M to be uninvertible only requires that some subset of rows is linearly dependent, which includes duplication, and makes a choice of $f(s)_j$ all the more important.

The easiest way to avoid this problem is by not taking character directly from the beginning but instead starting in the middle and take differently sized steps. Consider the following set:

$$S = \{\text{set_APPLE, set_APRICOT, get_APPLE, get_APRICOT}\}$$

In this case we take the 7th and 9th characters instead.

This solution boils down to finding an n dimensional array σ, such that when $f(s)_j$ is the $\sigma(j)$th character of s the resulting M is invertible. This search can be easily automated and done at whatever time S is know, which for many application is compile time.

Polynomial Methods

4.1 Interpolation

Given two n size arrays of numbers y_k and x_k their polynomial interpolation[1]is a $n-1$ degree polynomial $p(X)$ such that:

$$y_k = p(x_k)$$

This process boils down to finding coefficients a_k of p:

$$p(X) = \sum_{i=0}^{n-1} a_k X^k$$

This is already a kind of hash, where we hash the elements x_k to y_k.

This hash is a bit primitive, and evaluating a polynomials in the naïve way would be too expensive, but with some work they can be used as a base for better hashes.

4.1.1 Lagrange Interpolation

The Lagrange Method is the most direct way to interpolate some points into a polynomial, and also gives a nice closed form for the polynomial.

Given x_k define the Lagrange basis functions as:

$$L_j(X) = \prod_{\substack{j=0 \\ k \neq j}}^{n-1} \frac{X - x_k}{x_j - x_k}$$

[1]This type of polynomial interpolation is more precisely called Lagrange to distinguish it from Hermite interpolation which includes setting the fitting the derivative at x_k.

Here the $\neq$ under the $\prod$ means we skip the term where $k = j$. For example consider the $n = 3$ case with $\{x_0, x_1, x_2\}$:

$$\begin{aligned}
L_0(X) =& \frac{X - x_1}{x_0 - x_1}\frac{X - x_2}{x_0 - x_2} \\
L_1(X) =& \frac{X - x_0}{x_1 - x_0}\frac{X - x_2}{x_1 - x_2} \\
L_2(X) =& \frac{X - x_0}{x_2 - x_0}\frac{X - x_1}{x_2 - x_1}
\end{aligned}$$

The utility of these functions is the values they take on x_k:

$$\begin{aligned}
L_0(x_0) =& \frac{x_0 - x_1}{x_0 - x_1}\frac{x_0 - x_2}{x_0 - x_2} = 1 \times 1 = 1 \\
L_0(x_1) =& \frac{x_1 - x_1}{x_0 - x_1}\frac{x_1 - x_2}{x_0 - x_2} = 0 \times \frac{x_1 - x_2}{x_0 - x_2} = 0 \\
L_0(x_2) =& \frac{x_2 - x_1}{x_0 - x_1}\frac{x_2 - x_2}{x_0 - x_2} = \frac{x_2 - x_1}{x_0 - x_1} \times 0 = 0
\end{aligned}$$

(And likewise for $L_1(X)$ and $L_2(X)$).

We get the interpolation by multiplying these functions by y_k and summing:

$$p(X) = \sum_{k=0}^{n-1} y_k L_k(X)$$

This can be verified by evaluation:

$$\begin{aligned}
p(x_0) =& y_0 L(x_0) + y_1 L(x_0) + y_2 L(x_0) \\
=& y_0 \times 1 + y_1 \times 0 + y_2 \times 0 \\
=& y_0 \\
p(x_1) =& y_0 L(x_1) + y_1 L(x_1) + y_2 L(x_0) \\
=& y_0 \times 0 + y_1 \times 1 + y_2 \times 0 \\
=& y_1 \\
p(x_2) =& y_0 L(x_2) + y_1 L(x_2) + y_2 L(x_2) \\
=& y_0 \times 0 + y_1 \times 0 + y_2 \times 1 \\
=& y_2
\end{aligned}$$

This method is useful for lower values of n but expanding the Lagrange basis functions becomes costly as n increase. So a better method is needed.

Before showing the better method there's one cool thing the Lagrange Method does, it enables us to reuse previous work. For example say we have an interpolation p for x_k, y_k, and n but we want to add another element y and x Then we can get the updated interpolation $P(X)$ by:

$$P(X) = p(X) + (y - p(X)) \prod_{k=0}^{n-1} \frac{X - x_k}{x - x_k}$$

When $X = x_k$ the $\prod$ term is 0 and when $X = x$ it's 1 meaning the two $p(X)$s cancel, which I think is neat.

4.1.2 Vandermonde Matrix

We have previously seen that field operations are compatible with matrices. In particular that we have compute the inverse of non-singular matrices, so if we can express interpolation as a linear matrix equation we can reuse and efficient algorithm.

To this end define the Vandermonde Matrix[2] V as:

$$V(x_0, x_1, \ldots, x_n) = \begin{bmatrix} 1 & x_0 & x_0^2 & \ldots & x_0^n \\ 1 & x_1 & x_1^2 & \ldots & x_1^n \\ 1 & x_2 & x_2^2 & \ldots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \ldots & x_n^n \end{bmatrix}$$

It's utility comes from the fact we can write:

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \ldots & x_0^n \\ 1 & x_1 & x_1^2 & \ldots & x_1^n \\ 1 & x_2 & x_2^2 & \ldots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \ldots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

[2]Some authors define the Vandermonde Matrix as having the highest powers on the left or x_0 on the bottom. Such minor change don't change the overall structure of the application.

Which is precisely the type of matrix equation that Gaussian Elimination can easily invert.

The only loose end is under what conditions is the Vandermonde Matrix invertible. The closed from of Lagrange Basis Functions is useful here, observe that the functions are all well defined provided $x_m \neq x_n$ and once we have the functions interpolation directly follows. Since a $n-1$ degree polynomial is uniquely defined by its value at n points because Vandermonde and Lagrange are calculating the same polynomial we would also expect the matrix to be invertible when $x_m \neq x_n$.

This intuition can be made rigours by calculating the determinate of the Vandermonde Matrix. The full calculation can be found in the appendix, but cutting to the chase we have:

$$\det(V(x_0, x_1, \ldots, x_n)) = \prod_{0 \leq i \leq j \leq n-1} (x_j - x_i)$$

This result is called the Vandermonde Polynomial and has wider application that just inverting the Vandermonde Matrix but for our purposes it matches our intuition. The determinate is non zero when $x_n \neq x_m$ and zero when $x_n = x_m$, for some n and m.

4.2 Evaluation

An important application for hashing is improving performance, hence there efficient polynomial evaluation is a must. For historical reasons algorithms that evaluate polynomials are called schemes and in this section I will be convening three schemes.

4.2.1 Naïve Scheme

Let a_k be the kth order coefficient for an n degree polynomial. Then the naïve method of evaluating the polynomial at x involves evaluating the first n powers of x. Multiplying the powers by the coefficients then summing them together.
For example, consider evaluating:

$$2 + x + 3x^2 + x^3$$

at $x = 2$.

1. Calculate a list of powers: $x^0 = 1$, $x^1 = 2$, $x^2 = 4$, $x^3 = 8$.
2. Multiply by coefficients: $a_0x_0 = 2$, $a_1x^1 = 2$, $a_2x^2 = 12$, $a_3x^3 = 8$.
3. Sum all the results: $2 + 2 + 12 + 8 = 24$.

Assuming we evaluated the powers of x by multiplying the previous power by x instead of starting from scratch, this method takes $n-1$ multiplications to evaluate the powers, n multiplications for multiplying the coefficients by the powers, and $n - 1$ additions.

4.2.2 Horner's Scheme

Horner's scheme is an iterative polynomial evaluation scheme where we combine the multiplication for generating powers with the multiplication for the coefficients. Algebraically, it looks like this:

$$a_0 + a_1x + a_2x^2 + a_3x^3 = a_0 + x(a_1 + x(a_2 + x(a_3)))$$

Algorithmically, we initialize with:

$$y_0 = a_n$$

Then iterate with:

$$y_k = a_{n-k} + xy_{k-1}$$

There are n iterative steps and each step involves one multiplication and one addition. Reusing the naïve example we get:

1. Initialize: $y_0 = 1$
2. Iterate: $y_1 = a_2 + xy_1 = 3 + 2 \times 1 = 5$
3. Iterate: $y_2 = a_1 + xy_2 = 1 + 2 \times 5 = 11$
4. Iterate: $y_3 = a_0 + xy_3 = 2 + 2 \times 11 = 24$

Which gives the same result of 24 but for only three multiplications and three additions.

4.2.3 Estrin's Scheme

In terms of number of multiplications, Horner's Scheme is the best and one of the most common evaluation scheme for general polynomial evaluation. But there is one flaw. Every iterative step requires the previous to be completed, which really limits the ability to parallelize the algorithm.

Estrin's scheme converts an n degree evaluation at x into a $\lfloor n/2 \rfloor$ degree evaluation at x^2 using $\lceil n/2 \rceil$ additions and multiplications that can be done in parallel, algebraically:

$$a_0 + a_1 x + a_2 x^2 + a_3 x^3 = (a_0 + x a_1) + x^2 (a_2 + x a_3)$$

Algorithmically, we input an n-array a_k of coefficients along side x and output the $\lceil n/2 \rceil$-array b_k along side x^2 such that:

$$b_k = a_{2k} + x a_{2k+1}$$

And iterate until we have a 1-array with the value as it's only element.

1. Input: $[2, 1, 3, 1]$ and $x = 2$
2. Iterate: $[2 + 1 \times 2, 3 + 1 \times 2] = [4, 5]$ and $x = 4$
3. Iterate: $[4 + 5 \times 4] = [24]$ and $x = 8$

4.3 Application to Hashing

Minimal perfect hashing.

Advanced Encryption Standard

As alluded to in the introduction one use of the Rijndael field is the Advanced Encryption Standard.

The basis of AES is the Rijndael field and some matrix operations, meaning we inadvertently have all tools necessary to give an overview. Not sure I will implment this in C.

Appendix

A.1 Ring Theory 101

A ring is an algebraic structure that acts like matrix arithmetic. This means you can add, subtract, and multiply matrices as you would expect. But you can't necessarily divide or change the order or multiplication.

For some quick examples consider the following matrices:

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, B = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, C = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

You can add the matrices:

$$A + B = B + A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Negate the matrices:

$$-A = \begin{bmatrix} -1 & -1 \\ 0 & -1 \end{bmatrix}$$

And multiply matrices:

$$C(A + B) = CA + CB = \begin{bmatrix} 2 & 1 \\ 0 & 0 \end{bmatrix}$$

However the order of multiplication matter:

$$AB = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}$$

$$BA = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix}$$

And not every matrix is guaranteed an inverse (meaning you can't divide by it):

$$\det C = \begin{vmatrix} 1 & 0 \\ 0 & 0 \end{vmatrix} = 0$$

So when talking about fields why do we care about rings? Firstly all field are rings, even though division is not guaranteed in rings they don't forbid it, so understanding rings helps us understand fields. But secondly, because rings are more general then fields they can act as stepping stones. If we want to show something is a field we can show that it's a ring and then use our knowledge of rings to do the last two steps to show it's a field.

This reason is also why Ring Theory 101 is in the appendix, most casual readers will be OK assuming or deducing for themselves how division works. In this case just skip using rings as a stepping stone.

A.1.1 Commutative Rings

As already shown the order of multiplication may matter in rings. Rings where order doesn't matter, i.e. $AB = BA$, are called commutative. Commutative Rings are algebraic structures that act like integers. Where you can add, subtract, and multiply how would expect but still can't divide.

Importantly, because fields are commutative and rings all fields are also commutative rings.

Thinking about commutative rings as integers way will help with the coming sections, but the term commutative is used all over mathematics in general and abstract algebra in particular and is worth committing to memory.

A.1.2 Polynomial Rings

Given a commutative ring R we define define the ring of polynomials with coefficients in R as $R[X]$[1]. For example consider the commutative ring of integers $\mathbb{Z}$ then:

$$\{1 + 2X, X^2, 7\} \subset \mathbb{Z}[X]$$

[1] Also called polynomials **over** R.

These polynomials can be "formal" meaning we don't necessarily want or need to substitute in a value for X. All we really care about is the relationship between coefficients, for example that:

$$(a_0 + a_1X)(b_0 + b_1X) = a_0b_0 + (a_1b_0 + a_0b_1)X + a_1b_1X^2$$

This "formal" view also shows why we need R to be commutative, since if R is not commutative then we will get some annoying non terms when multiplying polynomials, for example:

$$(aX)(bX) = aXbX$$

Which doesn't necessarily equal abX^2 when R isn't commutative.

Quotient Polynomial Rings

We have already implicitly used the concept of a quotient ring when defining the ring of numbers less than 5. But to say it explicitly we take a ring, $\mathbb{Z}$, and a set within the ring, multiples of 5, and say two numbers of the original ring are equal if and only if their difference is in the set. Also written as the elements being equal **congruent** the set.

For example: The elements 2 and 7 of $\mathbb{Z}$ are equal congruent the multiples of 5, often shortened "congruent 5" or just "congruent" if the set is obvious from context.

This process only produces a ring when the set follows certain properties, called being a "two-sided ideal", which are outside the scope of this document. But all the multiples of a given element are often "two-sided ideals".

This idea also works for polynomial rings. Consider the quotient ring of $\mathbb{Z}[X]$ modulo $X^2 - 2$, detonated $\mathbb{Z}[X]/(X^2 - 2)$. The elements X^3 and $2X$ and congruent since:

$$X^3 = 2X + X(X^2 - 2)$$

From this example you may have observed two things:

1 All the elements of $\mathbb{Z}[X]/(X^2-2)$ are congruent some degree 1 or lower polynomial.

2 X^2 is congruent to 2.

So every element of $\mathbb{Z}[X]/(X^2 - 2)$ can be thought as equaling:

$$a + bX \text{ where } a \text{ and } b \text{ are integers and } X^2 = 2$$

Meaning we took $\mathbb{Z}$ and adjoined $\sqrt{2}$, the root of $X^2 - 2$, to it.

And this observation is mostly correct, and a big reason as to why we care about Polynomial Rings when talking about Fields. Firstly the only thing the observation missed is that we could have equally adjoined $-\sqrt{2}$ to $\mathbb{Z}$. This redundancy is related to the symmetries of $X^2 - 2$, and is studied in depth in Galois theory, but means we can only say we adjoined **a** root of $X^2 - 2$ not which one.

Secondly the ability to adjoin a new element to a field through quotient polynomial rings will be necessary to understand the Rijndael Field. But we need one more tool for figuring out when the elements of quotient polynomial rings have reciprocals.

A.1.3 Bézout's Identity and Domains

One of the most useful tools in showing a Ring is a Field is Bézout's Identity. Before stating the more general form, consider Bézout's Identity for integers:

> Let x and y be integers with the greatest common divisor d. Then for all integers r and s there exists k such that:
>
> $$xr + ys = kd$$
>
> Furthermore, there exists integers r' and s' such that $k = 1$, i.e.:
>
> $$xr' + ys' = d$$

Water Pouring Puzzle

It can be helpful to think in Bézout's Identity in terms of Water Pouring Puzzles. For those unfamiliar with these puzzles, you are given jugs of known sizes and are asked to figure out a sequence of has a specific amount of water at the end. For example, you are given a 2L and 3L jugs and are ask to produce 1L of water. You could fill the 3L jug then pour into the 2L jug until it's full, then discard the 2L, leaving 1L in the original jug. As required.

Bézout's Identity says that the water in the jugs must always be a multiple of the greatest common divisor of the jugs. For example if you had a 4L and 6L jug then the water in the jugs is always a multiple of 2L. Why? Because we only have the capacity to manipulate water in multiples of 2L, so after all steps we still have multiples of 2L left.

Bézout's Identity goes on to say that that there alway exits a sequence of moves that get us to the greatest common divisor of water. This isn't as intuitive to see as the previous statement, but see that you can remove multiples of the smaller jug from the larger till less water is left in the larger jug then the volume of the smaller. This effectively creates a new smaller jug with which we can repeat this process till the jugs are the same size, leaving the greatest common reminder.

Proving Reciprocation

To see how Bézout's Identity might might be useful in showing that elements of a ring have a reciprocal, and hence can be divided by, consider the example given in the introduction about integers less then 5. Constructing a multiplication table only works because 5 is a small number, consider a larger prime like 113 how do we show that a 18 has a reciprocal in this set? Well since 113 is prime and $18 < 113$ their greatest common divisor is 1. From Bézout's Identity there must exist numbers r and s such that:

$$113 \times r + 18 \times s = 1$$

Although not strictly necessary for this argument, sufficient r and s can be calculated[2] as -7 and 44, giving:

$$113 \times (-7) + 18 \times 44 = 1$$

This can be rewritten as:

$$18 \times 44 = 1 + 113 \times 7$$

Hence 18×44 has remainder 1 when divided by 113. Making 44 the reciprocal of 18 for the ring of integers less than 113.

[2] An efficient way to calculate r and s for the general case is called Euclid's Extended Algorithm. While outside the scope, if you understand this argument you are able to understand the algorithm and I suggest looking it up.

This argument works just as well for all non-zero integers less than 113 hence all non-zero integers have a reciprocal making this commutative ring a field. This argument also for any set of integers less than a prime number, hence all such rings are actually fields.

A similar argument shows that the ring of integers less than a given number n are fields **only** work for prime number. Otherwise there is a number m that's greatest common divisor with n, labeled d, is greater than 1. And through Bézout's Identity we know that for all r and s we have k such that:

$$nr + ms = kd > d > 1$$

Hence we can't divide by m and the ring isn't a field.

Application to Polynomial Rings

Rings where Bézout's Identity apply are called "Bézout Domains". Domains are algebraic structure in-between rings and fields where $ab = 0$ implies $a = 0$ or $b = 0$ and Bézout Domains are a subtype of domain. It's outside of scope to detail another type of algebraic structure, but it is important to say that Polynomial Rings are a type of Bézout Domain.

This means we can prove reciprocation in a Quotient Polynomial Rings in the same way we did with integers. Consider $\mathbb{Q}[X]/(X^2 - 2)$ from earlier discussion we know this ring is like taking $\mathbb{Q}$ and adjoining $\pm\sqrt{2}$ to it. But $\mathbb{Q}$ is a field and not a ring! Surly we should be able to recover division after adjoining $\pm\sqrt{2}$.

Proof: If x is the root of a degree 1 or less polynomial in $\mathbb{Q}[X]$ then x is rational since:

$$ax + b = 0 \to x = -\frac{b}{a}$$

Hence $X^2 - 2$ is relatively prime to all elements of $\mathbb{Q}[X]$ of degree 1 or less since all roots of $X^2 - 2$ are irrational. From Bézout's Identity this means that for any degree 1 or lower element p of $\mathbb{Q}[X]$ we can find elements $r(X)$ and $s(X)$ of $\mathbb{Q}[X]$ such that:

$$(X^2 - 2)r(X) + p(X)s(X) = 1$$

Meaning every element of $\mathbb{Q}[X]$ of degree 1 or less can be reciprocated. And since X^2 is degree 2 every element of $\mathbb{Q}[X]/(X^2-1)$ is congruent to an element of degree 1 or less. Hence every element of $\mathbb{Q}[X]/(X^2-2)$ is invertible, making it a field.

A.1.4 Conclusion

Being an appendix the main goal of Ring Theory 101 is to communicate key ideas around rings for the main text. These ideas are:

- Rings are more general fields where order of multiplication can matter but without division.
- Rings where order of multiplication doesn't matter are called commuting rings.
- Given a commuting ring R we can make a new ring $R[X]$ of polynomials with coefficients in that ring.
- Bézout's Identity is a great tool for finding out when an element of a ring has a reciprocal.
- Conceptually Quotient Polynomial Rings are like taking the base ring and adjoining a new element to it.

A.2 Adjoining a Number to a Field

$F(a)$ means the smallest field including both the field F and the number a and is referred as adjoining a to F. As suggested by Ring Theory 101's conclusion, a standard way to do this is to consider polynomials with coefficients in F, denoted $F[X]$, and find a polynomial $p(x) \in F[X]$ such $p(a) = 0$ consider the quotient ring of $F[X]/q(X)$, using Bézout's identity when necessary.

But it's really important not to confuse polynomials with coefficients in F, denoted $F[X]$, polynomials with coefficients in F evaluated at a, denoted $F[a]$, and the field F adjoined with $F(a)$. What makes this issue more confusing is that when a is the root of a polynomial in F[3] we do have $F[a] = F(a)$.

[3]This property is called a being algebraic over F

Without lost of generality let f be the minimal polynomial such that $f(a) = 0$. Let g be a different polynomial in $F[X]$. By the minimality of f either $g(a) = 0$ or f and g are relatively prime. If $g(a) = 0$ then $g = 0$ in $F[a]$ meaning it doesn't need a reciprocal, hence f and g are relatively prime, and hence from Bézout's identity we have polynomial r,s in $F[X]$ such that:

$$f(x)r(x) + g(x)s(x) = 1$$

But when we substitute in a we have:

$$g(a)s(a) = 1$$

Hence g has a reciprocal in $F[a]$ and from this, with a bit more legwork, we have $F[a] = F(a)$

But when a isn't algebraic this isn't true. Consider a number a that isn't the solution of some polynomial in F. Then $F(a) \neq F[a]$ because $a^{-1} \notin F$. For example there is no element $\pi^{-1} \notin F[\pi]$.

As much as I stress that $F[a]$ doesn't necessarily equal $F(a)$ the fact that for most contexts it does means confusion is likely. Its the missing apostrophe of field theory, as much as I warn you about it dont worry about it. Like missing apostrophise someone will tell you.

A.3 Generator Tables

For completeness sake I've included the index and power tables when 03 is the generator. In table look-up, the columns are lowest half byte and the row is the highest. For example the index of $A3$ is $6F$. In both case the lookup for 00 is arbitrary and chosen to simplify some algorithms.

Index:

	·0	·1	·2	·3	·4	·5	·6	·7	·8	·9	·A	·B	·C	·D	·E	·F
0·	00	FF	19	01	32	02	1A	C6	4B	C7	1B	68	33	EE	DF	03
1·	64	04	E0	0E	34	8D	81	EF	4C	71	08	C8	F8	69	1C	C1
2·	7D	C2	1D	B5	F9	B9	27	6A	4D	E4	A6	72	9A	C9	09	78
3·	65	2F	8A	05	21	0F	E1	24	12	F0	82	45	35	93	DA	8E
4·	96	8F	DB	BD	36	D0	CE	94	13	5C	D2	F1	40	46	83	38
5·	66	DD	FD	30	BF	06	8B	62	B3	25	E2	98	22	88	91	10
6·	7E	6E	48	C3	A3	B6	1E	42	3A	6B	28	54	FA	85	3D	BA
7·	2B	79	0A	15	9B	9F	5E	CA	4E	D4	AC	E5	F3	73	A7	57
8·	AF	58	A8	50	F4	EA	D6	74	4F	AE	E9	D5	E7	E6	AD	E8
9·	2C	D7	75	7A	EB	16	0B	F5	59	CB	5F	B0	9C	A9	51	A0
A·	7F	0C	F6	6F	17	C4	49	EC	D8	43	1F	2D	A4	76	7B	B7
B·	CC	BB	3E	5A	FB	60	B1	86	3B	52	A1	6C	AA	55	29	9D
C·	97	B2	87	90	61	BE	DC	FC	BC	95	CF	CD	37	3F	5B	D1
D·	53	39	84	3C	41	A2	6D	47	14	2A	9E	5D	56	F2	D3	AB
E·	44	11	92	D9	23	20	2E	89	B4	7C	B8	26	77	99	E3	A5
F·	67	4A	ED	DE	C5	31	FE	18	0D	63	8C	80	C0	F7	70	07

Power:

	·0	·1	·2	·3	·4	·5	·6	·7	·8	·9	·A	·B	·C	·D	·E	·F
0·	01	03	05	0F	11	33	55	FF	1A	2E	72	96	A1	F8	13	35
1·	5F	E1	38	48	D8	73	95	A4	F7	02	06	0A	1E	22	66	AA
2·	E5	34	5C	E4	37	59	EB	26	6A	BE	D9	70	90	AB	E6	31
3·	53	F5	04	0C	14	3C	44	CC	4F	D1	68	B8	D3	6E	B2	CD
4·	4C	D4	67	A9	E0	3B	4D	D7	62	A6	F1	08	18	28	78	88
5·	83	9E	B9	D0	6B	BD	DC	7F	81	98	B3	CE	49	DB	76	9A
6·	B5	C4	57	F9	10	30	50	F0	0B	1D	27	69	BB	D6	61	A3
7·	FE	19	2B	7D	87	92	AD	EC	2F	71	93	AE	E9	20	60	A0
8·	FB	16	3A	4E	D2	6D	B7	C2	5D	E7	32	56	FA	15	3F	41
9·	C3	5E	E2	3D	47	C9	40	C0	5B	ED	2C	74	9C	BF	DA	75
A·	9F	BA	D5	64	AC	EF	2A	7E	82	9D	BC	DF	7A	8E	89	80
B·	9B	B6	C1	58	E8	23	65	AF	EA	25	6F	B1	C8	43	C5	54
C·	FC	1F	21	63	A5	F4	07	09	1B	2D	77	99	B0	CB	46	CA
D·	45	CF	4A	DE	79	8B	86	91	A8	E3	3E	42	C6	51	F3	0E
E·	12	36	5A	EE	29	7B	8D	8C	8F	8A	85	94	A7	F2	0D	17
F·	39	4B	DD	7C	84	97	A2	FD	1C	24	6C	B4	C7	52	F6	01

A.4 Side-Channel

I've said in a few places that using generators to simplify field arithmetic makes the implementation susceptible to side-channel attacks. But how? And what is a side-channel attack?

A side-channel is something about the implementation of an algorithm or protocol that lets an attacker get information, rather than something inherently wrong with the algorithm or protocol itself.

The easiest to explain side-channel is the timing. Consider a guarded door and to get access you have to tell the guard a password. You tell the guard two different incorrect passwords but the guard rejects one immediately but waits till you're half way through for the second one. You might reasonably conclude that first password is wholly wrong but that the first half of the second one is correct. By doing this enough times and keeping track of the time the guard rejects the password you can probably figure out the password. You guessing the password has nothing to do with the cryptography of a password but the implementation by the guard, instead they should have waited till you've said the whole password before rejecting it, and hence is a side-channel attack.

Using a look-up table to compute field operations opens you up to a caching side-channel attack.

Because the look-up table is to large to be stored in the CPU registers it will be accessed from memory. Accessing the table from memory means it's likely to be cashed. The table being cached means someone with the ability to monitor cache access can see what field elements you're using, and maybe work backwards to get some of the string key that was hashed. Hence leaking the string.

This makes using look-up tables unsuitable for application where the string keys are sensitive information.

Before closing this topic I want to bring up that side-channels can include the physical properties of computers as well. Consider a naïve attempt at

stopping a timing attack where the guard waits to reject the password but is still noticeably paying less attention. An attacker can still guess the timing from when the guard stops paying attention. Similarly idling a computer till a set amount of time passes for all password validation attempts might cause a change in power consumption or a noticeable reduction in cooling. Hence even power-draw or acoustic properties of a computer are possible side-channels for attacks.

A.5 Matrix Arithmetic

A.5.1 Diagonalization

The description of Gaussian Elimination and the claim that it works with any invertible matrix may make some readers that half remember a linear algebra course nervous. This is because process described is similar to another process, diagonalization, which is not guaranteed to work just because the input matrix is invertible.

A diagonal matrix is one where all non-diagonal elements are zero and transcribed as Diag:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 7 \end{bmatrix} = \mathrm{Diag}(1, 3, 2, 7)$$

Such matrices are important they are the same as the matrix for the elementary row operation of scaling the rows, making matrix multiplication invoking them conceptually and computationally easy. For example:

$$\mathrm{Diag}(a_0, a_1, \ldots a_n)\,\mathrm{Diag}(b_0, b_1, \ldots b_n) = \mathrm{Diag}(a_0 b_0, a_1 b_1, \ldots a_n b_n)$$

Diagonalization is the process of taking a matrix M and finding a diagonal matrix D and invertible matrix P such that: [4]

$$MP = PD$$

[4] Often equivalently written as $M = PDP^{-1}$

And this process is important because it enables the simplification of calculations by canceling P's and utilizing diagonal matrix multiplication:

$$\begin{aligned}
M^3 =&(PDP^{-1})^3 \\
=&(PDP^{-1})(PDP^{-1})(PDP^{-1}) \\
=&PD(P^{-1}P)D(P^{-1}P)DP^{-1} \\
=&PD^3P^{-1} \\
=&P\,\mathrm{Diag}(a_k)^3P^{-1} \\
=&P\,\mathrm{Diag}(a_k^3)P^{-1}
\end{aligned}$$

But unfortunately diagonalization isn't guaranteed for invertible matrices. The conditions of when it works involves something called eigenvalues, which are outside the scope of this document. But what is concerning is how similar this looks to Gaussian Elimination, which given $MA = B$ involves finding an invertible P such that:

$$M'A = PMA = PB$$

Where M' is M after some elementary row operations. What is important to notice is that in diagonalization P is on the right side of M and the left side of D. But during Gaussian Elimination P is on the left side of both M and B. Since matrix multiplication isn't commutative this makes the processes very different and lays nerves to rest.

Readers interesting in learning more about diagonalization and it's uses are encouraged to first loop up eigenvalues and then "The Jordan Normal Form".

A.5.2 Vandermonde Polynomial

This proof of the determinant of the Vandermonde Matrix, the Vandermonde Polynomial, relies on the readers familiarity with three properties of the determinant.

1 If a row has only one non-zero element then the determinant is equal to that element times by the matrix with that row and column removed.

2 Multiplying a row by a constant multiplies the determinant by the same constant.

3 Adding a multiple of one row to another doesn't change the determinant.

For those familiar with these properties skip the next paragraph, for those not remember that geometrically the determinant is the signed n-dimensional volume of the parallelotope of the matrices column. You don't need to know what that means rigorously[5] but can think of it like getting the volume of a regular cube. In this case the first property is like how the volume of a cube is its depth times its cross-section, the second property is like how if you double a single cube's side length you double it's area, and the third property is like describing the cubes points relative to other points instead the origin.

The first step of finding the determinate is to subtract v_0 times the previous column from each column, this doesn't change the determinate:

$$\begin{vmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ 1 & x_2 & x_2^2 & \dots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{vmatrix} = \begin{vmatrix} 1 & x_0 - x_0 & x_0^2 - x_0x_0 & \dots & x_0^n - x_0x_0^{n-1} \\ 1 & x_1 - x_0 & x_1^2 - x_0x_1 & \dots & x_1^n - x_0x_1^{n-1} \\ 1 & x_2 - x_0 & x_2^2 - x_0x_2 & \dots & x_2^n - x_0x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n - x_0 & x_n^2 - x_0x_n & \dots & x_n^n - x_0x_n^{n-1} \end{vmatrix}$$

$$= \begin{vmatrix} 1 & 0 & 0 & \dots & 0 \\ 1 & x_1 - x_0 & (x_1 - x_0)x_1 & \dots & (x_1 - x_0)x_1^{n-1} \\ 1 & x_2 - x_0 & (x_2 - x_0)x_2 & \dots & (x_2 - x_0)x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n - x_0 & (x_n - x_0)x_n & \dots & (x_n - x_0)x_n^{n-1} \end{vmatrix}$$

Since there is a row with where the only non-zero element is 1, so the deter-

[5] And if you aren't familiar with these properties then this description alone probably doesn't help.

minant is the same if we removed that row and column.

$$\begin{vmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ 1 & x_2 & x_2^2 & \dots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{vmatrix} = \begin{vmatrix} x_1 - x_0 & (x_1 - x_0)x_1 & \dots & (x_1 - x_0)x_1^{n-1} \\ x_2 - x_0 & (x_2 - x_0)x_2 & \dots & (x_2 - x_0)x_2^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ x_n - x_0 & (x_n - x_0)x_n & \dots & (x_n - x_0)x_n^{n-1} \end{vmatrix}$$

Now observe that we can factor $(x_1 - x_0)$ from the top row, and $(x_2 - x_0)$ from the next, and so on:

$$\begin{vmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ 1 & x_2 & x_2^2 & \dots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{vmatrix} = \prod_{k=1}^{n}(x_k - x_0) \begin{vmatrix} 1 & x_1 & \dots & x_1^{n-1} \\ 1 & x_2 & \dots & x_2^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \dots & x_n^{n-1} \end{vmatrix}$$

Which is the nth dimensional Vandermonde polynomial in term of the $(n-1)^{\text{th}}$, hence induction gives the required result.

A.6 Hardware and Instruction Sets

A.6.1 CLMUL Instruction Set

Carry-less multiplication basically polynomial multiplication in GF(2).

A.6.2 AES Instruction Set